

MLIR/NPU Compiler Lecture 17

Fusion, Layout Propagation, and Transformer MLP Analysis

Topic: Transformer MLP fusion candidate analysis | Output: fusion/layout/cost contract

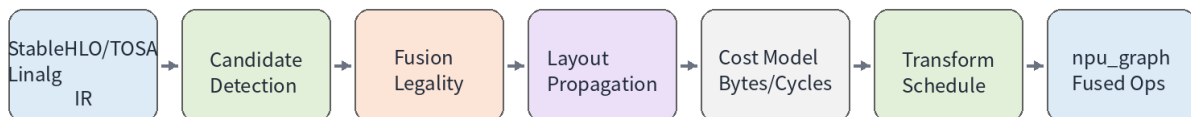
1. 강의 목표

- Fusion 을 단순 pattern rewrite 가 아니라 legality, layout, cost model, runtime ABI 를 포함한 compiler contract 로 이해한다.
- Transformer MLP 에서 MatMul + Bias + GELU, MatMul + Bias, SwiGLU elementwise fusion 후보를 분석한다.
- Layout propagation 을 통해 unnecessary transpose/repack/copy 를 줄이고, materialization boundary 를 명시한다.
- NPU SRAM/ACCUM live range, DMA traffic, tensorization tile shape 관점에서 fusion profitability 를 평가한다.
- FileCheck 와 negative diagnostics 로 fusion/layout propagation pass 를 검증하는 방법을 설계한다.

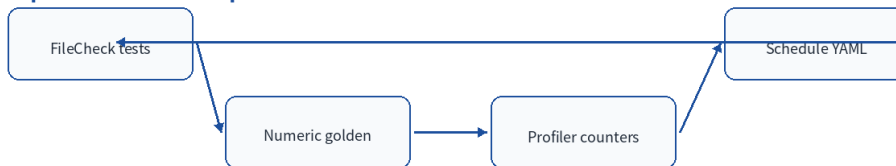
2. 17 강의 위치

이 강의는 14 강 Vector/Vectorization, 15 강 Transform Dialect, 16 강 Quantization 다음에 위치합니다. 지금까지는 연산을 NPU-friendly 형태로 표현하고 scheduling 하는 방법을 배웠고, 17 강에서는 여러 op 를 하나의 실행 단위로 묶을지, 어떤 layout 을 유지할지 결정합니다.

Lecture 17: Fusion + Layout Propagation Pipeline



Compiler feedback loop



Key output: fusion groups + propagated layout contracts + materialization points + NPU schedule plan

3. Fusion 의 정의: op fusion, kernel fusion, command fusion

- Op fusion: IR level 에서 producer/consumer 를 하나의 semantic op 또는 fused region 으로 표현한다.
- Kernel fusion: 여러 op 를 하나의 NPU kernel 로 lower 해 intermediate DRAM write 를 제거한다.
- Command fusion: command buffer 상에서 DMA/compute/barrier sequence 를 하나의 scheduling unit 으로 묶는다.
- MLIR pass 에서 가장 중요한 것은 어느 abstraction level 에서 fusion decision 을 내리는지 명확히 하는 것이다.

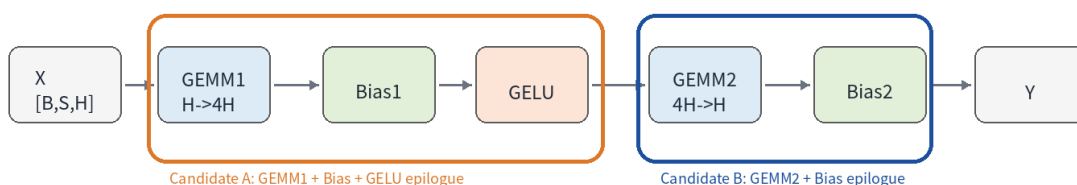
4. NPU 에서 Fusion 이 중요한 이유

관점	Fusion 이득	위험	Compiler 가 확인할 것
DRAM traffic	intermediate write/read 제거	layout repack 이 더 비쌀 수 있음	bytes saved > bytes materialized
SRAM reuse	tile-local epilogue 실행	live range 증가	SRAM/ACCUM footprint
Tensorization	native matmul tile 유지	fused epilogue 가 tile shape 를 깨뜨림	M/N/K legality
Quantization	requant/clamp fused 가능	rounding/saturation 차이	numeric contract
Runtime ABI	fewer commands	debug/profiling 복잡도 증가	command descriptor schema

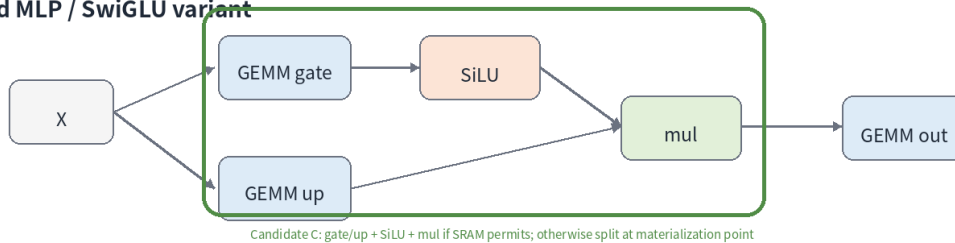
5. Transformer MLP Case Study

기본 MLP 는 일반적으로 $X \rightarrow \text{GEMM1} \rightarrow \text{Bias} \rightarrow \text{GELU} \rightarrow \text{GEMM2} \rightarrow \text{Bias}$ 구조입니다. LLM 계열에서는 SwiGLU 처럼 gate/up 두 projection 뒤 SiLU 와 multiply 를 수행하는 변형도 흔합니다. NPU compiler 는 각 후보를 fusion group 으로 묶되, SRAM footprint 와 layout materialization 비용을 동시에 계산해야 합니다.

Transformer MLP Fusion Candidates



Gated MLP / SwiGLU variant



6. Fusion Candidate Legality Matrix

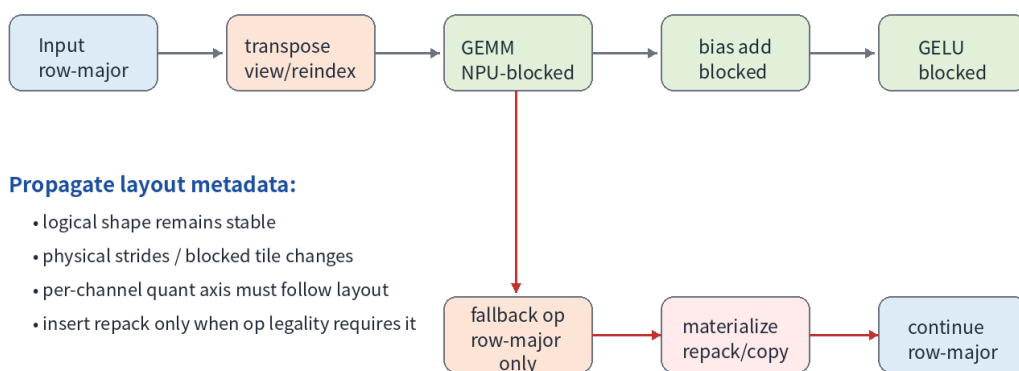
후보	필수 조건	Layout 조건	Quant 조건	Fallback
MatMul+Bias	bias broadcast axis == N	output layout supports epilogue	bias scale = lhs*rhs	separate bias op
MatMul+Bias+GELU	activation approximation accepted	blocked output ok	activation before/after requant defined	split after bias
SwiGLU	gate/up tile streamable	same tile layout on both branches	scale axis aligned	materialize one branch
Layout propagation	consumer accepts propagated layout	view/elementwise passthrough	quant axis preserved	layout_materialize
Weight prepack	static or cacheable weights	KxN packed layout	per-channel scale follows N	runtime prepack

7. Layout Propagation 핵심

- Logical shape 와 physical layout 을 분리해서 생각한다. Compiler IR type 은 tensor shape 를 유지하되 layout attribute/encoding 으로 physical view 를 표현한다.
- Elementwise op 는 대부분 layout-preserving 입니다. 따라서 producer 의 blocked layout 을 consumer 까지 밀어 넣는 것이 유리합니다.
- Reduction, transpose, gather/scatter, CPU fallback, function ABI boundary 는 materialization point 가 될 가능성이 높습니다.
- Per-channel quantization 의 channel axis 가 layout transformation 이후에도 정확히 대응되는지 검증해야 합니다.

Layout Propagation Graph

Goal: push one physical layout through cheap ops and materialize only at unavoidable boundaries.



NPU compiler contract:

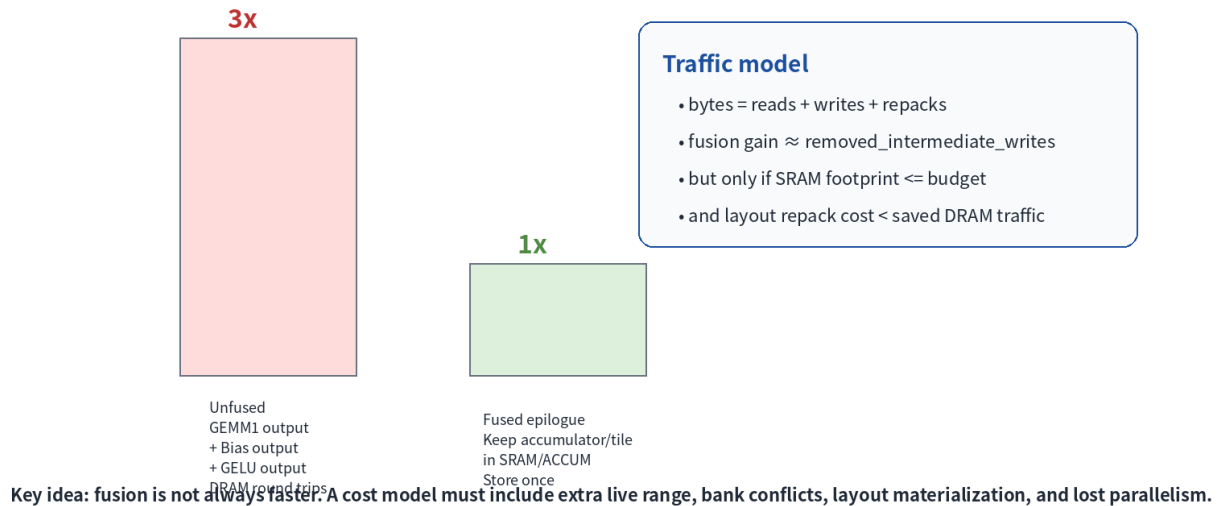
update fusion legality, DMA descriptors, vector maps, and runtime ABI.

8. Cost Model: fusion 이 이득인지 계산하기

- Compute cycles: MAC count / effective TOPS. Fused epilogue 가 tensor core pipeline 에 끼워지는지 확인합니다.
- DMA cycles: DRAM bytes / bandwidth. Intermediate write/read 가 줄어드는 만큼 이득입니다.
- Materialization cost: layout repack/copy 가 발생하면 fusion 이득을 상쇄할 수 있습니다.
- SRAM footprint: inputs, weights, output tile, accumulator, epilogue temporaries, double buffer 를 모두 포함합니다.
- Parallelism: fusion 으로 live range 가 길어져 다른 core/tile scheduling 을 막으면 오히려 느려질 수 있습니다.

Memory Traffic Reduction by Fusion

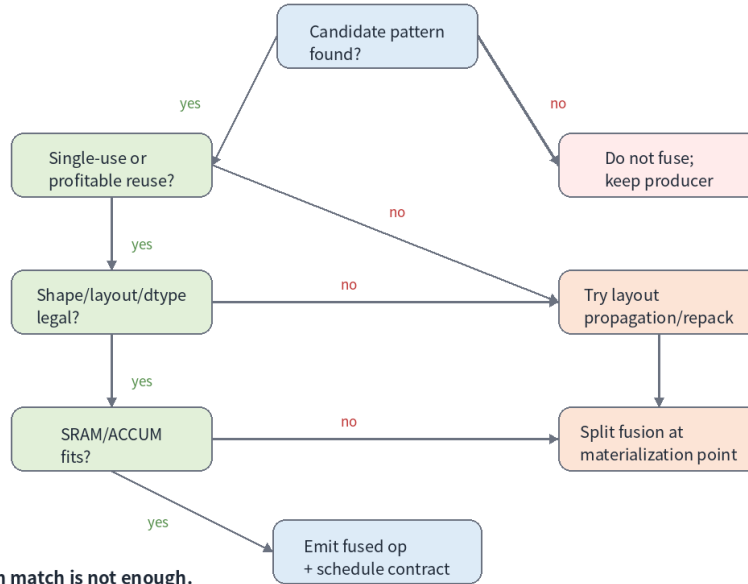
Example: MLP intermediate H=4096, expansion=4, sequence tile=128, FP16 activations.



9. Pass 설계

- npu-detect-fusion-candidates: use-def chain 과 linalg structured op 를 기반으로 후보 group 을 태깅합니다.
- npu-propagate-layout: preferred layout 을 producer/consumer 방향으로 전파하고 materialization point 를 명시합니다.
- npu-evaluate-fusion-cost: SRAM, bandwidth, tile utilization, repack cost 를 계산합니다.
- npu-emit-fused-graph-ops: npu_graph.matmul_epilogue 또는 npu_graph.mlp_fused 같은 target contract op 를 생성합니다.
- npu-verify-fusion-contract: shape/layout/dtype/quant/ABI metadata 가 모두 존재하는지 검증합니다.

Fusion + Layout Propagation Decision Tree



Code review rule: pattern match is not enough.
Every fused op needs a legality contract and cost justification.

10. Example IR: before/after

아래는 fused op 가 표현해야 하는 핵심 attribute 입니다. 실제 upstream MLIR op 가 아니라 NPU graph dialect 설계 예시입니다.

```

%h = "npu_graph.matmul_epilogue"(%x, %w1_packed, %b1) {
  epilogue = "bias_gelu_tanh",
  lhs_layout = "MxK_row",
  rhs_layout = "KxN_blocked_32x16",
  out_layout = "MxN_blocked_16x16",
  accumulator = "fp32",
  fusion_id = "mlp.gemm1.bias.gelu"
} : (...) -> tensor<128x16384xf16, #npu.layout<"MxN_blocked_16x16">>
  
```

11. FileCheck 검증 전략

- Positive test: fused op 가 생성되고 epilogue/layout attribute 가 존재하는지 확인합니다.
- Negative test: unsupported shape, multiple consumers, quant axis mismatch 에서 fusion 이 발생하지 않거나 diagnostic 이 나오는지 확인합니다.
- CHECK-NOT 으로 unfused bias/activation op 가 제거되었는지 확인하되, canonicalization 에 의해 이름이 바뀔 수 있는 부분은 느슨하게 검사합니다.
- materialization op count 를 검사해 layout propagation 이 copy 폭발을 만들지 않는지 regression test 를 만듭니다.

12. 실습

실습	내용	산출물
A	Transformer MLP before IR 에서 fusion 후보 표시	candidate matrix

B	blocked layout 을 elementwise chain 에 propagation	layout propagation graph
C	SRAM footprint 와 traffic saving 계산	cost model table
D	fused npu_graph op 와 materialization point 작성	after pseudo MLIR
E	positive/negative FileCheck 작성	regression test skeleton

13. Common Mistakes

- Pattern 만 맞으면 fusion 하는 실수: shape/dtype/layout/quant legality 를 반드시 분리해서 확인해야 합니다.
- Layout propagation 후 quant axis 를 업데이트하지 않는 실수: per-channel scale 이 잘못 적용될 수 있습니다.
- SRAM footprint 계산에서 accumulator 와 double buffer 를 빠뜨리는 실수: 실제 hardware 에서 tile spill 이 발생합니다.
- Function boundary ABI 를 무시하는 실수: runtime descriptor 와 consumer layout 이 불일치합니다.
- FileCheck 에서 fused op 존재만 확인하는 실수: unfused op 부재와 materialization count 까지 확인해야 합니다.

14. References

- MLIR Linalg Dialect: <https://mlir.llvm.org/docs/Dialects/Linalg/>
- MLIR Transform Dialect: <https://mlir.llvm.org/docs/Dialects/Transform/>
- MLIR Data Layout Modeling: <https://mlir.llvm.org/docs/DataLayout/>
- MLIR MemRef Dialect: <https://mlir.llvm.org/docs/Dialects/MemRef/>
- MLIR Transform Tutorial Chapter 0: <https://mlir.llvm.org/docs/Tutorials/transform/Ch0/>